

1. Introduction

The following project aimed to develop an NLP-based platform to analyze whether text messages sent by individuals can be classified as suicidal or non-suicidal. In a time when the internet is such an integral part of daily life, many who are struggling with suicidal thoughts may show glimpses of that online. Current studies suggest that those who are suicidal express their feelings to some degree over the internet before their suicide (Balt, Merelle, Robinson, 2023). The rise in online clues before a person's suicide led to the development of this project: a text classifier tool that analyzes user-input text and gives a classification as suicidal or non suicidal and a confidence score. The tool uses multiple NLP models, like a classical Logistic Regression, LSTM with attention, and BERT. The output gives a comparison of these three approaches. A Streamlit application was created to make an effective platform to present these models.

The overall work was shared between the group members in order to complete the project effectively. The group consisted of Yeji, Anusha, and Ritu. They were each responsible for developing one of the models that were going to be used in the project. I was then responsible for taking all of their completed model code and creating a demo application that could allow for these models to be tested and showcased.

2. Description of individual work

I was responsible for making the Streamlit application that took all of the models and made them one unified product that could be tested and demonstrated. I had previously not used Streamlit, so I had to familiarize myself with the program and how I would build the application. The algorithm was designed based on the concept that we had in mind to test. Since we were developing a program that used each model to test if a text was suicidal, I first thought of the best way to demonstrate this. This led to the development of an initial program that allowed the user to select a model and then input a text to see if it was classified as suicidal or not. These models were from the code that my other group members produced. After this initial production, a discussion was held with the team to determine the specific vision and goals we were trying to demonstrate. We decided that there should only be one output and not multiple options to select models. This allowed the user to compare the results of the models for the same input to make a more informed decision based on all the models.

3. Work in Detail

Once the final layout of the application was determined, the next step was to code the final version of the program. I will go through the portions of the code in order to show in detail the purposes of each part of the code.

```
IMPROVED_TFIDF = "improved_baseline_tfidf.pkl"
IMPROVED_MODEL = "improved_baseline_lr.pkl"

BERT_MODEL_DIR = "models/bert_base_uncased_single"

LSTM_MODEL_PATH = "saved_model/lstm_attention_model.pt"
LSTM_VOCAB_PATH = "saved_model/vocab.pkl"
LSTM_LABEL_PATH = "saved_model/label_encoder.pkl"
```

I first had to download all of the files of the models that were going to be used in the application. This was important because I wanted to make sure the models being used were the same ones that were developed by my teammates. That's why I made the pathways to each of the models in order to call them later in the code.

The next part of the code was creating the loading and predicting functions for each of the models. These functions were based on the codes made by my teammates to ensure that the models were being properly displayed in the presentation. An example of these functions can be seen below.

```
def load_improved():
    tfidf = joblib.load(IMPROVED_TFIDF)
    model = joblib.load(IMPROVED_MODEL)
    return tfidf, model

def predict_improved(text: str): 1 usage
    tfidf, model = load_improved()
    X = tfidf.transform([text])
    pred = model.predict(X)[0]
    proba = model.predict_proba(X)[0]

    classes = model.classes_
    suicide_idx = None
    for i, c in enumerate(classes):
        if str(c).lower() == "suicide":
            suicide_idx = i
            break
    if suicide_idx is None:
        suicide_idx = 1 if len(classes) > 1 else 0

    proba_suicide = proba[suicide_idx]

    return str(pred), float(proba_suicide)
```

Each of these functions is designed to be called for a user input to put in, and give the output result for each model.

The final part of the code is designing the Streamlit UI to actually demonstrate all of the models being used. This was the final step, as the models were already input and callable. I chose a simple design to make it easily understandable by a user who doesn't have experience with coding languages. The layout was designed to have a simple title and a text input box that would allow the user to input their desired text. Then there is a button that the user pressed to allow the models to run. Lastly, there is a table that shows the classification of each model, the name of the model, and the confidence value. The code below shows the beginning of the UI design.

```
st.set_page_config(page_title="NLP Suicide Detection", layout="wide")

st.title("Suicidal Text Detection - DATS 6312")

st.write("Enter text below and click the button to see predictions from different models.")

user_text = st.text_area("Input text:", height=150, placeholder="Type the message you want to classify...")
```

This code shows how the UI was developed.

I then had to deploy the app through Google Cloud Platform so it works on any computer and from any source. This was done by integrating the code into GCP and creating a cloud application for it. I had to create an extensive requirements.txt folder and a Dockerfile in order to make sure the program ran properly. Lastly, I had to troubleshoot many issues for the application to work.

4. Results

The results of my part of the project can be seen in the created UI of the project. Below is multiple examples of the program being run.

Suicidal Text Detection — DATS 6312

Enter text below and click the button to see predictions from different models.

Input text:

Life is feeling very hopeless.

Run models

Results

	Model	Prediction	Confidence
0	Improved Baseline LR	suicide	0.9998
1	BERT	suicide	0.9984
2	LSTM + Attention	suicide	1.0000

Figure 1- Application being run for the text,” Life is feeling very hopeless”.

Suicidal Text Detection — DATS 6312

Enter text below and click the button to see predictions from different models.

Input text:

Life is feeling very happy and I have great friends.

Run models

Results

	Model	Prediction	Confidence
0	Improved Baseline LR	non-suicide	0.4659
1	BERT	non-suicide	0.0012
2	LSTM + Attention	non-suicide	0.3144

Figure 2- Application being run for the text,” Life is feeling very happy, and I have great friends”.

Both of these examples show the application being used and giving the results of the different models. In order to make sure these values were consistent with the code of my teammates, I ran the same text in the terminal using the programs of my teammates directly. They would always give the same values, which showed me the application was running correctly. I also ran many different tests using various amounts of phrases in order to test the limitations of each of the models. My teammates developed each of the programs, but they did not get to test the models extensively with user inputs. This program allowed me to test each of the models very efficiently, and it was discovered that the BERT model struggled with information texts, and it would give a poor classification in those scenarios. This was then further analyzed by my groupmates.

5. Summary and Conclusions

In summary, the results of my section is a user interface that properly shows the goals of our project. We wanted to develop a tool that allowed people to input text to classify it as suicidal or not. This application I developed, allows for that to be done. Through the process of this project, I learned a lot about developing a Streamlit UI that presents the contents of a project. I had previously never done any visualization through Streamlit, and this allowed me to learn all about the process. I also learned a lot about file management and environment settings, as taking all of the code files from my groupmates and adding them together was a difficult process. They each used different versions and had different necessary files for their individual models to run. I needed to mediate all of these in order to have a script that could manage all of these differences. One potential area of improvement could be to make a tool that can analyze large corpora that have multiple users' messages. For example, if this could become a type of scanner that goes through message boards, it could make it much more efficient to determine who needs help.

6. Code Analysis

$$(152-33)/(150+23)=0.688$$

7. References

- Balt, E., Mérelle, S., Robinson, J. et al. Social media use of adolescents who died by suicide: lessons from a psychological autopsy study. Child Adolesc Psychiatry Ment Health 17, 48 (2023). <https://doi.org/10.1186/s13034-023-00597-9>
- <https://www.kaggle.com/datasets/nikhileswarkomati/suicide-watch/data>